

MANIPULATOR DIFFERENTIAL KINEMATICS

Benjamin Ignacio Olmos Retamales
FICG, Carrera de Ingeniería Civil Electrónica
Universidad de Católica del Norte, Antofagasta, Chile

I. Abstract

This work presents the replication and analysis of a computational program focused on the kinematic modeling of the Panda robotic arm. The program was developed using Python and the Robotics Toolbox, SpatialMath, and NumPy libraries. First, elementary transformations involving rotations and translations were implemented using the Elementary Transform (ET) framework. These transformations were then combined to construct an Elementary Transform Sequence (ETS) representing the different components and joints of the Panda robot. The program also evaluates the robot's forward kinematics for a specific joint configuration by calculating the resulting homogeneous transformation matrix. The obtained transformation is compared with the results provided by the `fkine()` and `eval()` methods available in the Robotics Toolbox. Finally, the Panda robot model is generated and represented graphically using the selected joint configuration. This process allows the theoretical concepts of homogeneous transformations, spatial transformations, and forward kinematics to be related to their computational implementation. The analysis demonstrates how specialized Python tools can be used to represent and evaluate the kinematic behavior of a robotic manipulator.

II. Introducción

Hoy en día, el estudio de las señales cerebrales (EEG) es súper importante en la ingeniería para entender cómo funciona el cerebro y ayudar a detectar problemas de salud. El gran desafío es que estas señales son muy débiles y sensibles, por lo que siempre vienen llenas de ruido, como la interferencia de los cables eléctricos de la habitación o los mismos movimientos de la persona. Antes era muy difícil procesar estos datos porque se necesitaba software muy caro. Pero hoy existen herramientas gratuitas y de código abierto en Python. Una de las más conocidas es BioSPPy, una biblioteca que ya viene con funciones listas para limpiar y analizar datos médicos de forma rápida.

El objetivo de este trabajo es instalar BioSPPy, probar su código fuente con una señal cerebral simulada y ver si realmente funciona bien. Con esto queremos aprender cómo se aplican los filtros matemáticos en la vida real y demostrar que podemos hacer proyectos de ingeniería biomédica de buena calidad usando software libre.

III. Marco Teórico

Un filtro digital es una herramienta matemática que procesa una señal digital para eliminar el ruido (quitando interferencias en

audio, llamadas o en este caso señales médicas). Al limpiar, mejorar y seleccionar la información útil dentro de una señal es posible analizar de mejor manera una señal.

Existen 2 tipos de filtros, los FIR (respuesta finita al impulso) y IIR (respuesta infinita al impulso), están diferenciados por su función.

FIR = Sistemas estables que calculan la salida utilizando valores actuales y pasadas de la entrada del sistema, este filtro es el más potente respecto a la precisión del filtrado.

$$y[n] = \sum_{k=0}^N b_k \cdot x[n-k]$$

IIR = A diferencia del anterior sistema, este usa retroalimentación ($y[n] = x[n] + ay[n]$), por lo que es un filtro más eficiente, respecto al anterior, sin embargo un poco menos preciso.

$$y[n] = \sum_{k=0}^M b_k \cdot x[n-k] - \sum_{m=1}^N a_m \cdot y[n-m]$$

Para este trabajo se utilizará el filtro FIR.

Por otro lado, un algoritmo de detección es un conjunto de reglas y cálculos programados para identificar, reconocer y encontrar elementos o eventos específicos dentro de una señal.

Por otro lado los encefalogramas poseen diversos canales asignados según el n° de cátodos conectados al paciente, con los cuales pueden monitorear la actividad cerebral de cada uno. Los espectros cerebrales se analizan mediante distintas bandas, las cuales representan una frecuencia distinta de la actividad cerebral.

- **Bandas Delta (δ) y Theta (θ) [Frecuencias lentas]:**

Representan el estado de desconexión y calma.

En la vida real: La onda Delta (0.5 - 4 Hz) domina cuando estamos en el sueño más profundo. La onda Theta (4 - 8 Hz), que aparece en tus gráficos con mucha fuerza, representa el estado de somnolencia, la meditación profunda o cuando estás a punto de quedarte dormido.

- **Banda Alpha (α) – Lower y Higher [Frecuencia media]:**

Representa el estado de relajación.

En la vida real: Domina en el rango de 8 a 13 Hz. Se activa con mucha fuerza cuando estás despierto, pero sentado tranquilo, relajado y especialmente **cuando cierras los ojos**. Como en tus gráficos las bandas Theta y Alpha son las que tienen los números más altos en el eje Y (potencia), significa que tu simulación representa a una persona descansando plácidamente.

- **Banda Beta (β) [Frecuencia rápida]:**

Representa el estado de pensamiento activo y concentración.

En la vida real: Va de los 13 a los 30 Hz. Se dispara cuando estás despierto resolviendo un examen de ingeniería, analizando un problema, hablando en público o cuando sientes estrés y ansiedad. En tus gráficos esta banda es casi invisible (1e-5), lo que demuestra matemáticamente que el cerebro analizado no estaba haciendo ningún esfuerzo mental.

- **Banda Gamma (γ) [Frecuencia ultra rápida]:**

Representa el procesamiento de información al máximo nivel o ruidos musculares.

En la vida real: Va de los 30 a los 100 Hz. Se activa cuando el cerebro conecta diferentes ideas a la vez (por ejemplo, cuando tienes un momento de genialidad o "¡Eureka!"). Sin embargo, en la ingeniería médica, los valores en Gamma muchas veces representan simplemente pequeños ruidos causados por el paciente al morder o mover los músculos de la cara.

IV. Implementación

Primeramente comenzaré creando el entorno de trabajo mediante la consola de comandos del sistema (cmd, anaconda prompt o directamente en la terminal de VS Code).

```
(base) C:\Users\yoelc>conda create --name env_BioSPPy|
```

Luego se de crear el entorno de trabajo y activarlo correctamente, instalaré las librerías necesarias para poder ejecutar el código, la librería principal es BioSPPy, matplotlib y numpy.

```
Nº1> pip install BioSPPy matplotlib
```

Una vez realizado esto, ahora podemos comenzar a analizar cada una de las líneas de código, para poder analizar la señal seleccionada, en este caso un encefalograma.

```
import numpy as np
from biosppy.signals import eeg
```

Se iniciará cargando las librerías necesarias, en este caso BioPSy y numpy (para los arreglos lineales), una vez importadas las librerías necesarias, procederá a definir las variables necesarias para el modelo que representa el encefalograma (se usaron 2 canales para simplificar la visualización, en un encefalograma real cada canal representa un electrodo conectado a la cabeza del paciente, por lo que se pueden llegar a monitorear hasta 64 canales).

```
tasaMuestreo = 250 # Hz
duracion = 10
t = np.arange(0, duracion, 1 / tasaMuestreo)
canales = 2
```

Con las variables definidas podemos generar una señal que emule la actividad cerebral (estas señales poseen mucho ruido, es por eso que se usará np.random), “archivoEeg” crea una matriz de ceros la cual será llenada por la señal con ruido antes de ser filtrada.

```
np.random.seed(42)
ArchivoEeg = np.zeros((len(t), canales))
for i in range(canales):
    ArchivoEeg[:, i] = np.sin(2 * np.pi * 7 * t) + np.random.normal(0, 0.5, len(t))
```

Además, como se puede observar en la anterior foto, se utilizó un ciclo “for” con el largo del n° de canales, este ciclo “for” es el encargado de generar las ondas cerebrales y sumarles el ruido, que posteriormente se almacenará en la matriz “archivoEeg”, generando de esta manera nuestra señal.

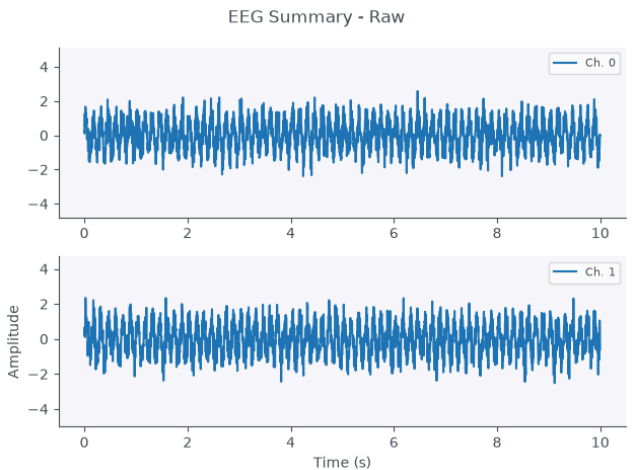
Ahora con nuestra señal definida, pasaremos a la canalización principal del código fuente, en este caso “salida”, la cuál será la que utilice la librería BioSPPy mediante la función eeg.eeg(). En este caso le entregaremos la matriz guardada (archivoEeg) y le diremos a que velocidad fueron tomados los datos de la señal (sampling_rate) y por último “Show=True”, para poder visualizar los gráficos interactivos generados que representan las señales generadas.

```
salida = eeg.eeg(signal=ArchivoEeg, sampling_rate=tasaMuestreo, show=True)
```

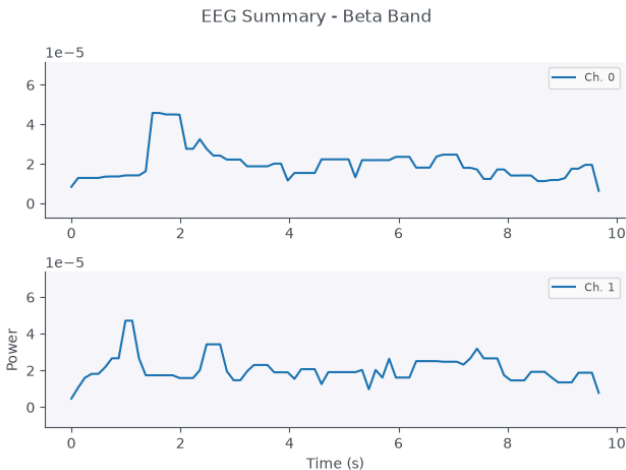
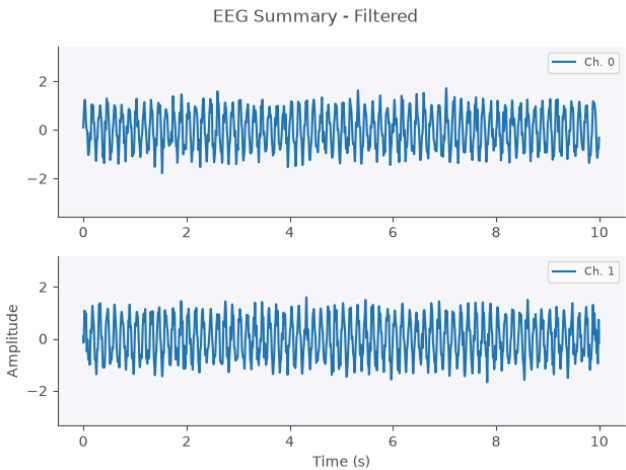
La función “eeg.eeg()” retornará la señal filtrada, la señal de la banda Alpha, Beta, Gamma, Theta y un

V. Resultados

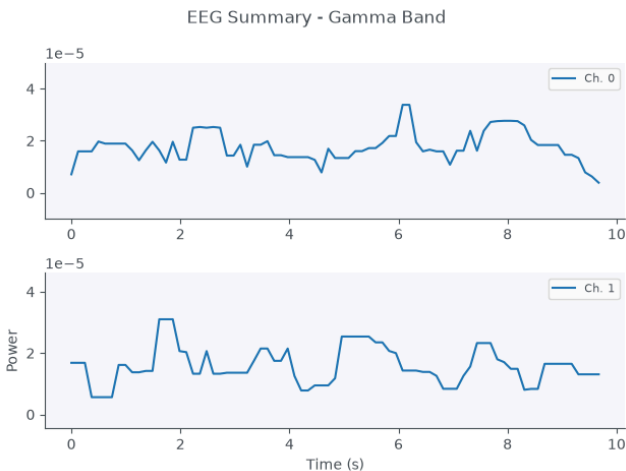
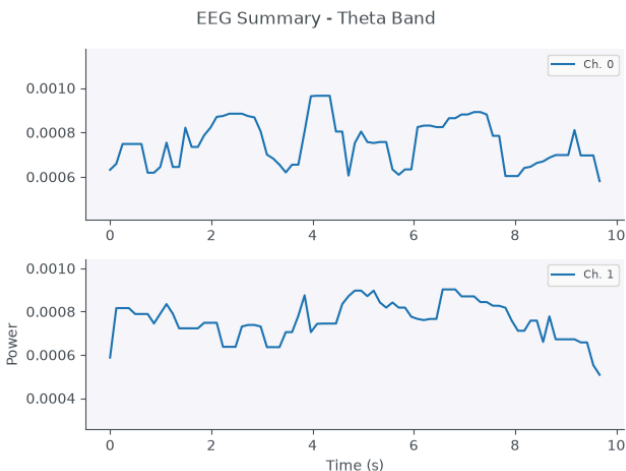
Como se explicó anteriormente, variable “archivoEeg” contiene una matriz que almaceno la señal con ruido que emula la actividad de un cerebro, en este caso la señal obtenida es la siguiente:



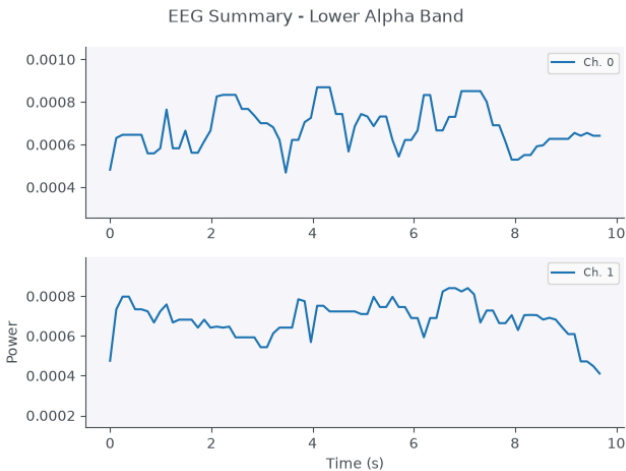
Posteriormente, este señal fue filtrada, procedimiento real y necesario para analizar con exactitud el comportamiento cerebral de un paciente y realizar un diagnóstico correcto, en este caso se obtuvo la siguiente gráfica:



Otra de las salidas que retorna la función son theta, gamma, beta y alpha, en este caso se obtuvo el siguiente resultado para la señal de las bandas:



Como las potencias de Theta y Lower Alpha son muchas veces más grandes que las de Beta y Gamma, el análisis matemático del código demuestra que el sujeto se encuentra en un estado de relajación profunda, meditación o descanso con los ojos cerrados. No hay un pensamiento activo o estrés (lo que dispararía la banda Beta).



Actualmente esta librería ha sido de gran ayuda para el área de investigación de la neurociencia, en estudios de las fases del sueño, desarrollo de relojes inteligentes que lean tus signos vitales, para sistemas de biometría y seguridad. BioSPPy es una de las librerías esenciales que todo ingeniero debería de conocer si quiere apuntar al área de la salud como especialización de ingeniero electrónico.

